

SPRINT 10 · REDES NEURONALES · S10.1

Fundamentos de redes neuronales

La dirección de TurisData oye hablar de redes neuronales en todas partes y pregunta si debe sustituir su modelo de cancelaciones. Primero hay que entender qué es una red por dentro.

La pregunta del reto

¿Una red neuronal mejora al aprendizaje automático clásico, y a qué precio?

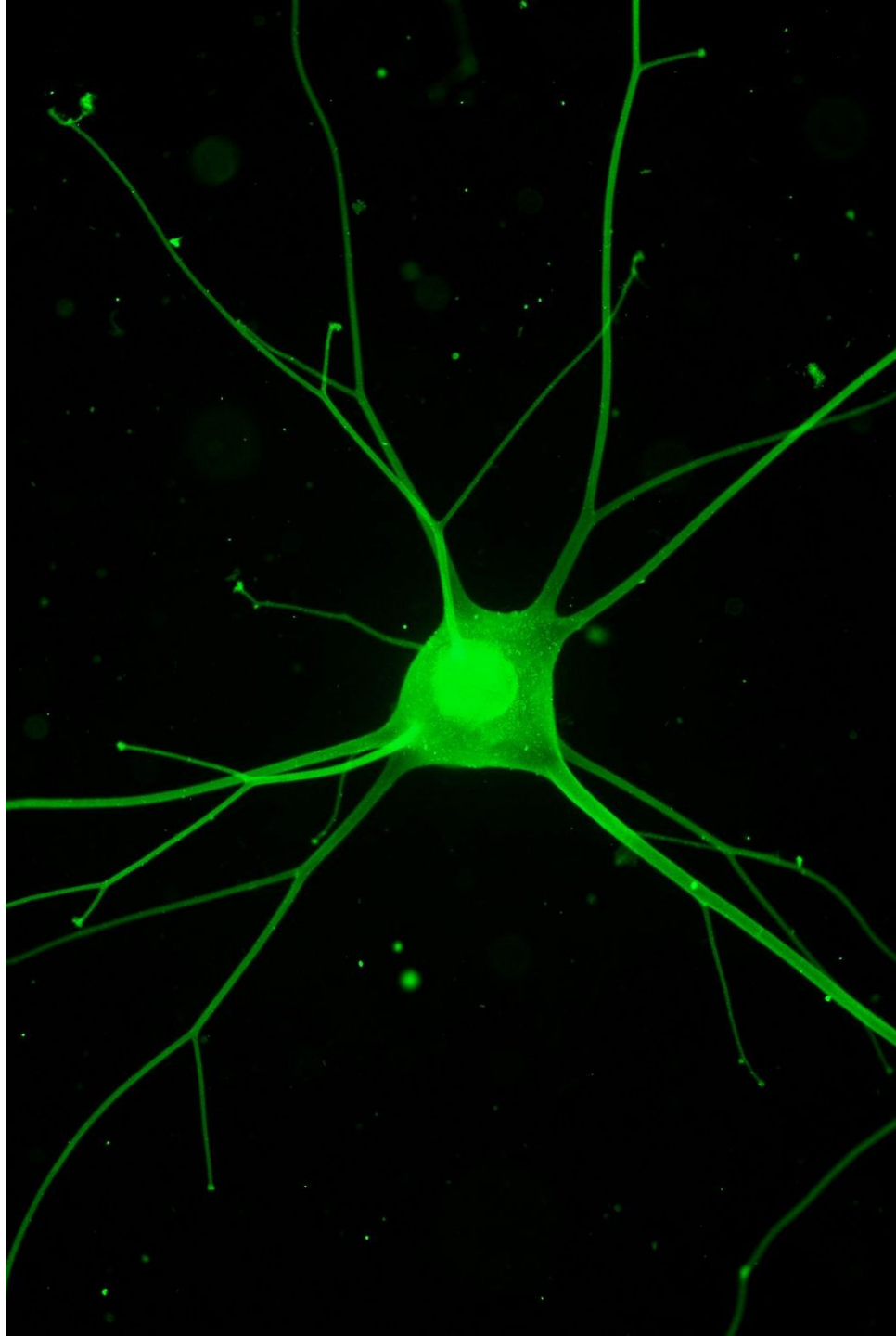
- Quiere evidencias con el coste incluido, no una opinión.
- Un resultado honesto de «no compensa» es tan válido como un «sí».
- Esta semana: entender la red (hoy), programarla en Keras, regularizarla y compararla con el campeón del Sprint 8.

La neurona: tres pasos

1. Multiplica cada entrada por un peso (cuánto le importa).
2. Suma todo y añade un sesgo (bias).
3. Pasa el resultado por una función de activación (un «interruptor suave»).

```
def neurona(x, pesos, sesgo):                # Ejercicio 1
    return sigmoide(np.dot(x, pesos) + sesgo)
```

Es la misma suma ponderada de la regresión logística: una red es muchas de estas apiladas.



Funciones de activación

Sin activación, apilar capas no sirve de nada: una suma de sumas ponderadas sigue siendo un modelo lineal. La activación introduce curvas.

Función	Salida	Uso típico
Sigmoide	entre 0 y 1	capa final de clasificación binaria
ReLU	0 si negativo, z si positivo	capas ocultas

```
def relu(z): return np.maximum(0, z)                # Ejercicio 2
def relu_derivada(z): return (z > 0).astype(float)
```

Una neurona ReLU apagada ($z \leq 0$) no aprende en esa muestra.

Un problema que una línea no resuelve

Dos «anillos» de puntos, uno dentro del otro: clientes de dos tipos que no se separan con una recta. La regresión logística se queda en el azar.

- La red del cuaderno: 2 entradas → capa oculta (ReLU) → 1 salida (sigmoide).
- Una capa = muchas neuronas a la vez = una multiplicación de matrices:

$$Z1 = X \cdot W1 + b1 \;\rightarrow\; A1 = \text{relu}(Z1) \;\rightarrow\; Z2 = A1 \cdot W2 + b2 \;\rightarrow\; A2 = \text{sigmoide}(Z2)$$

Aprender = descenso del gradiente (otra vez)

```
def paso_gradiente(parametros, gradientes, tasa):          # Ejercicio 3
    return {k: parametros[k] - tasa * gradientes[k]
            for k in ("W1", "b1", "W2", "b2")}
```

adelante → atrás (gradientes por retropropagación) → actualizar. Cada vuelta completa a los datos es una época.

Es el mismo bucle que programaste en el Sprint 5 con la regresión, con más pesos.

¿Cuántas neuronas hacen falta?

- El cuaderno entrena redes con 1, 2 y 8 neuronas ocultas: con 1 no separa los anillos; con 8 sí.
- Ejercicio 4: precision_val de la red de 8 y mejora_sobre_lineal frente a la logística.
- Mini-reto: 4 neuronas con tasa 0,01 durante 1500 épocas (red_lenta): ¿por qué le cuesta tanto? ¿Y con una tasa demasiado alta?



Ahora, a practicar

Lo que viene a continuación está en tu repositorio.

CUADERNO	Una red neuronal con solo NumPy (sprints/S10/notebooks/S10_01_red_desde_cero)	40 min